

EngAdda

VTU Engineering Notes

C PROGRAMMING

Student-friendly notes • Visual learning • Exam revision • Lab support

For VTU First-Year Engineering — foundation topics aligned to the 2025-scheme course listings

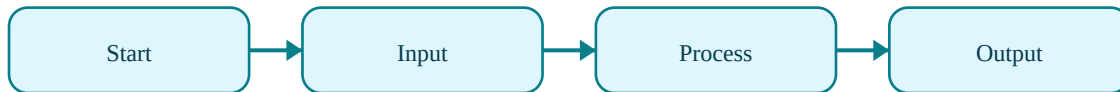
How to use these notes

Read the concept, study the example, and type the program yourself. Course codes and module order may vary by branch, so compare with your college's official VTU syllabus before exams.

Covered: problem solving, C basics, operators, decisions, loops, arrays, strings, functions, pointers, structures, files, lab practice and viva.

1. Problem Solving and Flowcharts

Algorithm = ordered steps to solve a problem. **Flowchart** = diagram of those steps.



Example: Add two numbers → Start → Read a,b → sum=a+b → Display sum → Stop.

Symbol	Meaning
Oval	Start / End
Parallelogram	Input / Output
Rectangle	Processing
Diamond	Decision

2. Structure of a C Program

```
#include <stdio.h>

int main(void)
{
    int a = 10, b = 20;
    printf("Sum = %d", a + b);
    return 0;
}
```

Remember: #include adds a header, main() is the starting function, statements usually end with ; and return 0 means successful completion.

3. Variables and Data Types

Type	Use	Format
int	Whole numbers	%d
float	Decimal values	%f
double	Higher precision	%lf
char	One character	%c

```
int age = 18;
float marks = 87.5f;
char grade = 'A';
```

Identifier rules: start with a letter or _, no spaces, no keywords, and C is case-sensitive.

4. Input and Output

```
int age;
printf("Enter age: ");
scanf("%d", &age);
printf("Age = %d", age);
```

scanf() usually needs & before ordinary variables because it receives their address.

5. Operators

Category	Operators	Example
Arithmetic	+ - * / %	a % 2
Relational	< > <= >= == !=	a >= b
Logical	&& !	a>0 && b>0
Assignment	= += -= *= /=	a += 5

Integer division: 5/2 gives 2; 5.0/2 gives 2.5.

6. Decisions

```
if (marks >= 40)
printf("Pass");
else
printf("Fail");
```

```
switch (choice)
{
case 1: printf("Add"); break;
case 2: printf("Subtract"); break;
default: printf("Invalid");
}
```

Use if-else for conditions/ranges. Use switch for fixed cases. break prevents unwanted fall-through.

7. Loops

```
for (int i=1; i<=5; i++)
printf("%d ", i);
```

```
int i=1;
while (i<=5)
{
printf("%d ", i);
i++;
}
```

for = known count; while = condition checked first; do-while = body runs at least once.

8. Arrays and Strings

An array stores same-type values. Indexing starts at 0.

```
int marks[5] = {80, 75, 91, 66, 88};  
printf("%d", marks[0]);
```

A string is a character array ending with \0.

```
char name[20] = "Gautam";  
printf("%s", name);
```

String functions: strlen(), strcpy(), strcat(), strcmp(). Include string.h.

9. Functions

```
int add(int x, int y)  
{  
    return x + y;  
}  
int main(void)  
{  
    printf("%d", add(4, 5));  
    return 0;  
}
```

Functions improve reuse, readability, testing and debugging. C normally passes arguments by value.

10. Pointers

A pointer stores a memory address. & gives an address; * accesses the value at that address.

```
int a=10;  
int *p=&a;  
*p=25;  
printf("%d", a);
```

11. Structures

A structure groups different data types under one name.

```
struct Student
{
    int usn;
    char name[30];
    float marks;
};
struct Student s={1, "Gautam", 85.5};
```

Use . with a structure variable and -> with a structure pointer.

12. Basic File Handling

```
FILE *fp=fopen("data.txt", "w");
if (fp != NULL)
{
    fprintf(fp, "Hello C");
    fclose(fp);
}
```

Modes: r = read, w = write/overwrite, a = append. Check NULL and close files.

13. Important Lab Programs

1. Temperature conversion
2. Calculator using switch
3. Prime check
4. Factorial
5. Fibonacci series
6. Largest array element
7. Linear search
8. Matrix transpose
9. String operations
10. GCD and LCM using functions
11. Student/employee structure
12. Addition using pointers

14. Viva Quick Revision

Question	Answer
What is C?	A procedural, general-purpose programming language.
Where does execution start?	At main().
What is a compiler?	Translates source code into machine/object code.
What is an array?	Same-type elements under one name.
What is a pointer?	A variable storing an address.
What is \0?	Null character ending a string.

Exam tip: write algorithm → code → sample input/output. Check semicolons, format specifiers, loop conditions and array limits.

EngAdda • Learn simply. Code confidently.